DYNAMIC RAG DOCUMENT INTELLIGENCE & BENCHMARKING SYSTEM

A Fair Comparison of Document Chunking Strategies

Prepared by
Naveen Kumar S

1. Abstract

This project implements a document ingestion and retrieval benchmarking workflow for Retrieval-Augmented Generation (RAG). It parses multiple document formats into structured blocks, applies six chunking strategies, creates embeddings, indexes chunks with FAISS, and evaluates retrieval using a fixed benchmark question set. The Streamlit dashboard supports document inspection, chunk comparison, retrieval exploration, benchmarking, and diagnostics.

The supplied OpenCode summary reports a six-document, 22-question benchmark and 89 passing tests. Scores and test counts are transcribed from that summary and have not been independently verified for this report.

2. Problem Statement and Objectives

RAG quality depends on whether document chunks preserve enough context while remaining focused for retrieval. Large chunks may dilute relevance; small chunks may separate facts from their context. This project compares strategies under consistent experimental conditions.

- Parse PDF, DOCX, Markdown, TXT, HTML, and CSV documents into structured blocks.
- Implement six interchangeable chunking strategies and preserve source metadata.
- Generate embeddings and build FAISS indexes for retrieval.
- Support dense, BM25, and hybrid retrieval.
- Evaluate Recall@k, MRR, nDCG@k, context completeness, and efficiency.
- Provide Streamlit, command-line benchmarking, and CSV/JSON exports.
- Support retrieval evaluation without a paid LLM API.

3. Technology Stack

Area	Technology
Language and interface	Python 3.11, Streamlit
Embeddings	sentence-transformers/all-MiniLM-L6-v2 (384 dimensions)
Vector indexing	FAISS (faiss-cpu), normalized vectors with IndexFlatIP
Document parsing	PyMuPDF, python-docx, BeautifulSoup/lxml, pandas, text/Markdown readers
Keyword and hybrid retrieval	rank-bm25 and Reciprocal Rank Fusion
Tokenization	tiktoken cl100k base with whitespace fallback
Optional answer generation	OpenAI-compatible API or Ollama
Testing and data	pytest, NumPy, pandas, JSON, CSV

4. System Workflow

- 1. Ingestion — identify file type and route it to the appropriate parser.
- 2. Parsing — extract typed blocks and preserve page, section, table, and source metadata.
- 3. Chunking — apply one or more strategies using a common configuration.

- 4. Embedding and indexing — embed chunks and build strategy-specific FAISS indexes.
- 5. Retrieval — retrieve relevant chunks using the selected retrieval mode and top-k.
- 6. Evaluation — run identical benchmark questions and calculate metrics.
- 7. Reporting — display rankings and charts and export CSV/JSON results.
- 8. Optional generation — answer using retrieved evidence and source citations.

5. Dynamic Parsing

A parser router chooses a format-specific parser and returns structured blocks rather than flattening the document into one string. Metadata may include source ID, file name, content type, page number, section path, table ID, parser name, and extraction warnings. PDF parsing retains page information; DOCX parsing retains headings and tables; HTML extraction removes irrelevant page elements; CSV parsing associates values with column headers. OCR is optional and requires a separately installed Tesseract executable for scanned PDFs.

6. Six Chunking Strategies

Strategy	Description
A. Fixed-size	Splits content into configured token windows with optional overlap; baseline strategy.
B. Sentence-based	Groups complete sentences within a token budget.
C. Structural	Uses headings, paragraphs, lists, tables, and code blocks as boundaries.
D. Recursive	Applies progressively smaller separators until chunks fit.
E. Semantic	Uses sentence-embedding similarity to identify topic changes.
F. Hybrid/dynamic	Combines structural boundaries with adaptive splitting and merging.

All strategies produce consistent chunk metadata, allowing the resulting indexes and retrieval performance to be compared.

7. Retrieval and Evaluation

The application supports dense FAISS retrieval, BM25 keyword retrieval, and optional hybrid retrieval using Reciprocal Rank Fusion. A fair comparison keeps the corpus, questions, embedding model, retrieval mode, and top-k identical across strategies.

Metric	Purpose
Recall@k	Measures relevant evidence retrieved within the top-k results.
MRR	Measures the rank of the first relevant result.
nDCG@k	Measures ranked retrieval quality using graded relevance.
Context completeness	Measures how many required facts appear in

	retrieved context.
Citation accuracy	Measures correct citations when generation evaluation is enabled; otherwise N/A.
Efficiency	Tracks chunk sizes, duplicates, stage timings, and index size.

8. Reported Benchmark Results

The supplied summary describes corpus northwind-v1 with six documents and 22 hand-labelled questions. The reported setup uses all-MiniLM-L6-v2, dense FAISS retrieval, top-k = 5, target chunk size 400 tokens, maximum 700 tokens, and overlap 60 tokens.

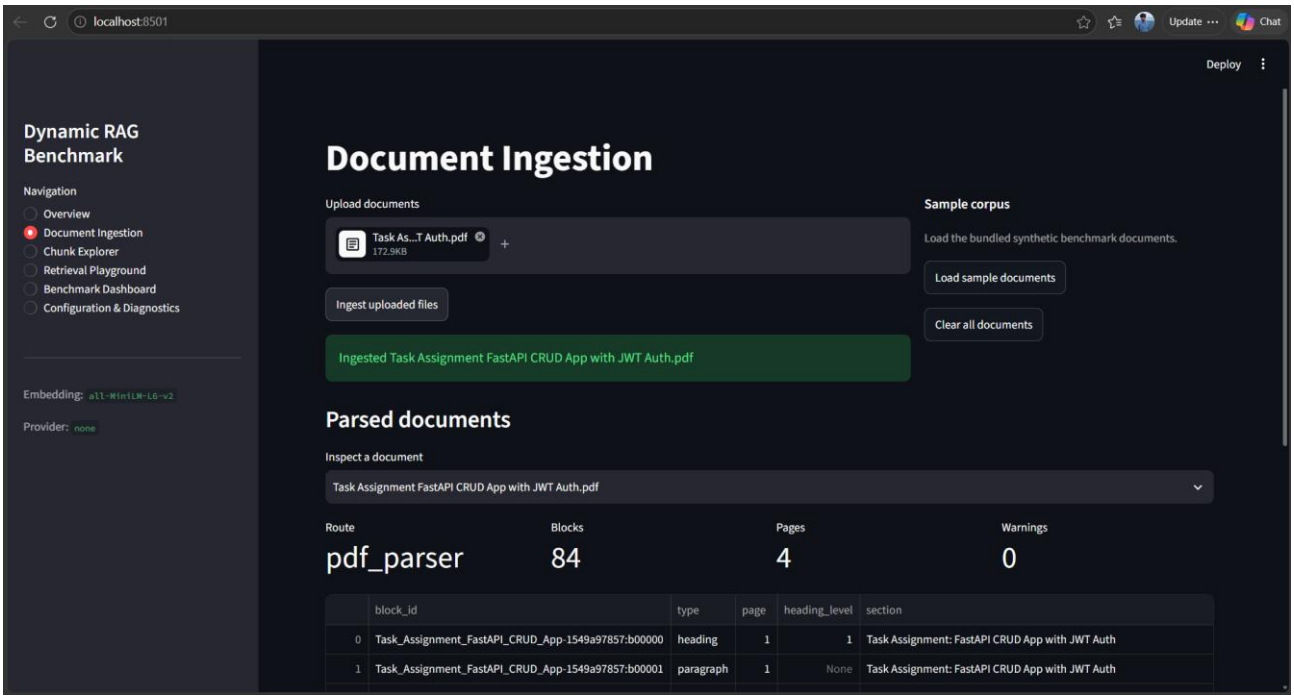
Rank	Strategy	Overall	Recall@5	MRR	Context completeness
1	Sentence-based	0.955	0.977	0.911	0.977
2	Structural	0.955	0.977	0.911	0.977
3	Semantic	0.955	0.977	0.911	0.977
4	Fixed-size	0.927	1.000	0.782	1.000
5	Recursive	0.924	1.000	0.773	1.000
6	Hybrid/dynamic	0.915	0.932	0.860	0.955

These are results reported by the supplied OpenCode summary; reproduce the benchmark in the application before treating them as independently verified.

9. Streamlit Application Screenshots and Explanation

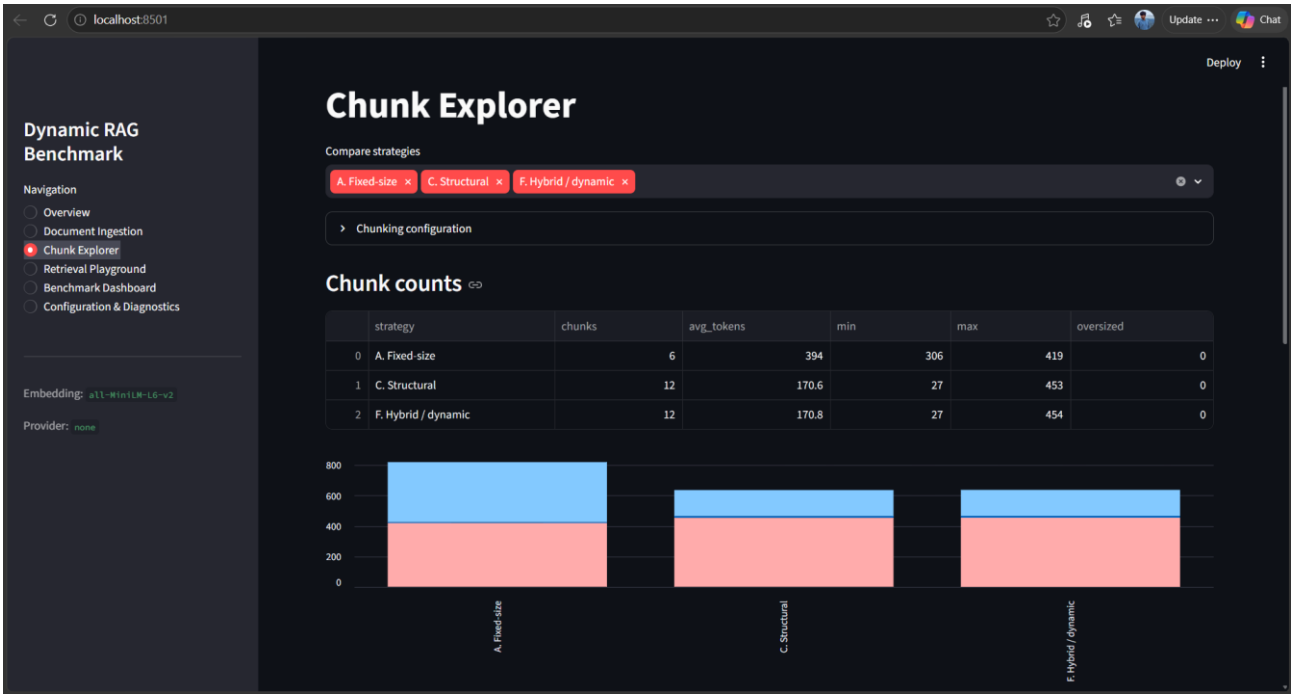
The following figures show selected screens from the running application, followed by explanations of the visible functions.

Figure 1. Document Ingestion — successful PDF parsing



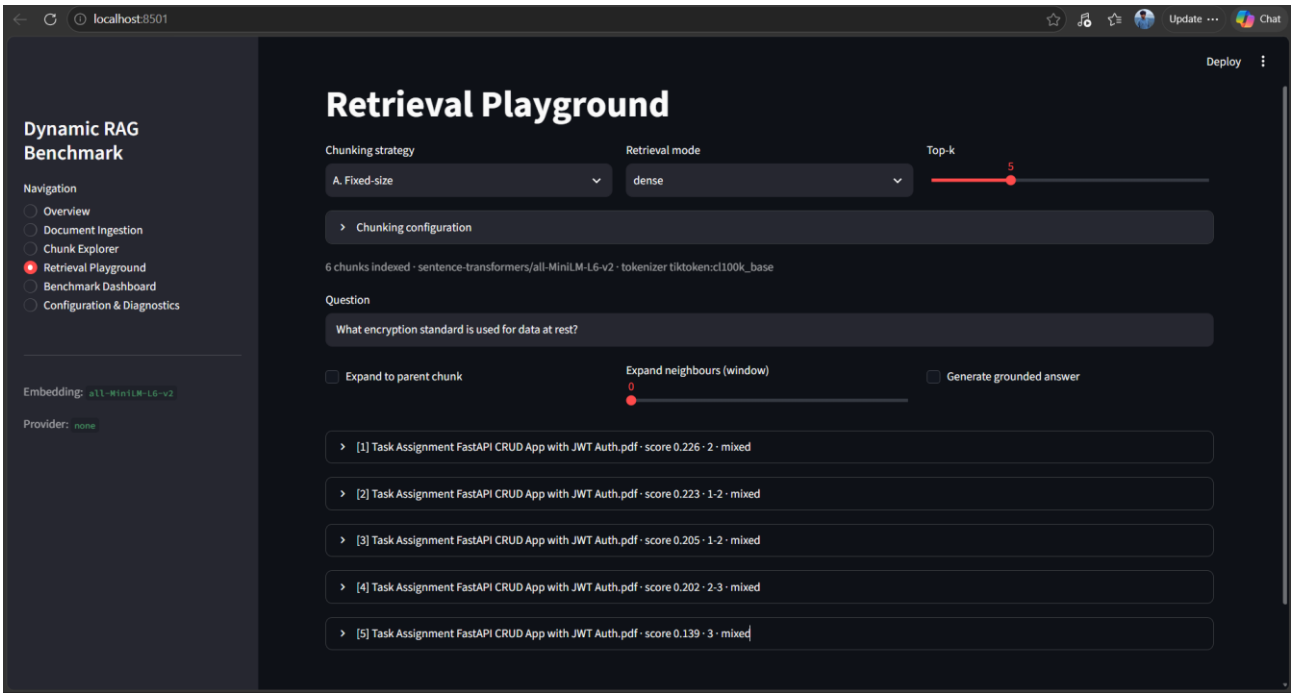
The uploaded FastAPI CRUD assignment PDF was routed to the PDF parser. The page reports 84 parsed blocks across 4 pages and zero extraction warnings. The parsed-block table exposes block IDs, content types, page numbers, heading levels, and section metadata for inspection.

Figure 2. Chunk Explorer — comparing chunking strategies



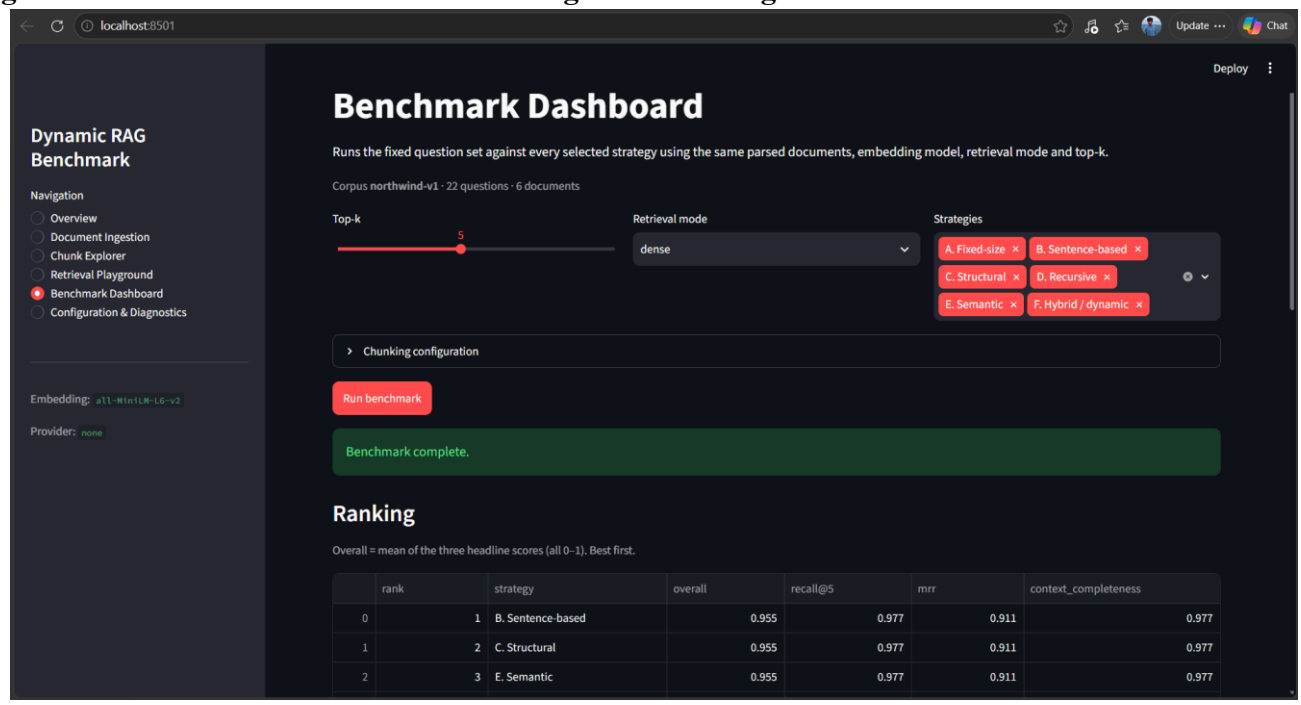
The Chunk Explorer compares fixed-size, structural, and hybrid/dynamic chunking. The table and chart show chunk counts and token-size statistics, helping users inspect how different boundary rules affect the resulting chunks.

Figure 3. Retrieval Playground — inspecting retrieved evidence



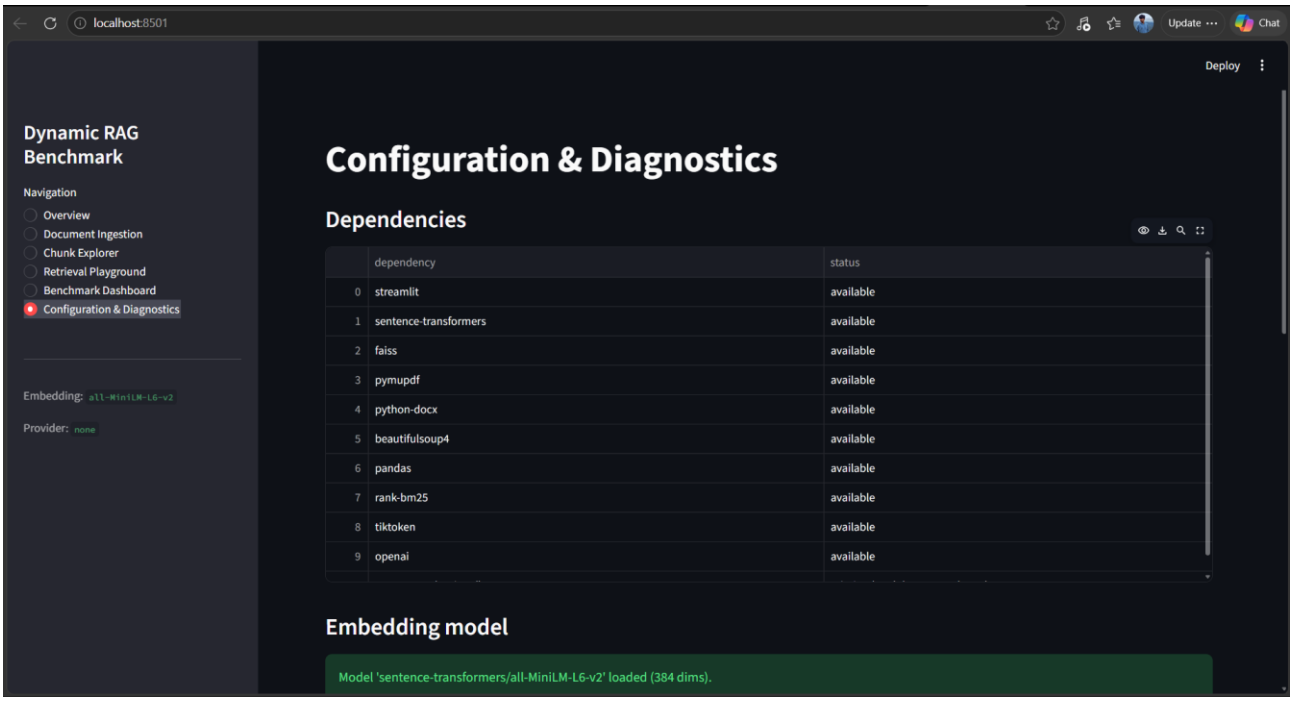
The user can choose a chunking strategy, retrieval mode, and top-k, then enter a question. Ranked passages display similarity scores and source information. The example asks which encryption standard is used for data at rest. Retrieval can be inspected without enabling an LLM.

Figure 4. Benchmark Dashboard — ranking the six strategies



The dashboard runs the same benchmark questions against selected strategies using consistent settings. The screenshot shows corpus northwind-v1, 22 questions, 6 documents, dense retrieval, and top-k = 5. The completed run displays a leaderboard with overall score, Recall@5, MRR, and context completeness.

Figure 5. Configuration & Diagnostics — checking dependencies



The diagnostics screen lists availability of key packages such as Streamlit, Sentence Transformers, FAISS, document parsers, BM25, and tokenization support. It also displays the embedding-model status, helping distinguish optional dependency issues from core pipeline problems.

10. Discussion

- Sentence-based, structural, and semantic strategies have the highest reported overall scores on the bundled corpus.
- Fixed-size and recursive strategies report perfect Recall@5 but lower MRR, showing that finding relevant evidence is not the same as ranking it first.
- Fine-grained chunks can improve ranking but may increase chunk count and index size.
- Results depend on the corpus and configuration; one corpus is not enough to establish universal superiority.

11. Testing and Quality Assurance

The supplied OpenCode summary reports 89 tests passing and one optional real-model integration test skipped unless RUN_EMBEDDING_TESTS=1 is enabled. Test areas reportedly include parsers, chunkers, metadata, embeddings, dense/BM25/hybrid retrieval, metric calculations, benchmark execution, exports, leaderboard ordering, and grounded-answer citations. These results were not independently rerun during report preparation.

12. Limitations

- OCR requires a separately installed Tesseract binary and is optional.
- Citation-accuracy scoring is unavailable in retrieval-only mode.
- The reported benchmark uses one bundled corpus; more corpora are needed for broader conclusions.
- Reported scores and test counts require independent reproduction.

13. Future Scope

- Benchmark multiple corpora and study cross-corpus generalization.
- Explore GPU acceleration and scalable FAISS index types.
- Tune chunk size and overlap systematically.
- Add answer faithfulness metrics using a local LLM.
- Improve OCR and layout-aware table extraction.
- Add PDF/HTML report exports and historical benchmark comparisons.

14. Conclusion

The system brings document parsing, adaptive chunking, embedding-based indexing, retrieval, and evaluation into one workflow. Its key contribution is comparing six chunking strategies under consistent conditions while inspecting retrieval quality and efficiency. The supplied results favor sentence-based, structural, and semantic chunking on the bundled corpus, but additional runs are needed to establish generalizability.

15. Project Repository

GitHub: https://github.com/naveenkumar-s05/Dynamic-RAG-Benchmark

The repository contains the source code, application modules, configuration files, and project documentation for the Dynamic RAG Document Intelligence and Benchmarking System.